

IBEROMAP

Manual de Migracion de Prototipo a Produccion

GUIA DE PASO A PRODUCCION: INTEGRACION DE BACK-END, BASE DE DATOS Y PERSISTENCIA REAL

Proyecto:

IberomapAngular (Angular v21)

Repositorio GitHub:

<https://github.com/polarsama/iberomap-angular>

Naturaleza del Sistema:

Guia de ingenieria para transicionar de un ambiente mock/in-memory a una arquitectura cliente-servidor real.

Generado en: Mayo de 2026 . Bogota D.C.

1. Introduccion y Naturaleza del Prototipo

El proyecto IberomapAngular es una aplicacion web interactiva desarrollada sobre el framework Angular v21. Este sistema esta disenado para la gestion, seguimiento y autoevaluacion de los programas academicos en el marco de la obtencion o renovacion de registros calificados y acreditaciones institucionales.

IMPORTANTE: El estado actual de este software corresponde estrictamente a un **PROTOTIPO FRONTEND**. La aplicacion **NO** esta conectada a un servidor back-end real ni a una base de datos centralizada de produccion. Toda la interactividad, el inicio de sesion de los usuarios, la creacion de registros, modificaciones de programas academicos y asignacion de docentes se ejecutan localmente en el navegador web del usuario.

Para lograr una experiencia de usuario completamente funcional y fluida sin depender de servicios externos, el sistema implementa mecanismos de simulacion en memoria y persistencia temporal. A continuacion se detallan los pilares de este comportamiento de prototipo:

* Persistencia en LocalStorage:

Los tokens de autenticacion y los datos del usuario activo se guardan en el almacenamiento local del navegador (localStorage). Esto simula la sesion persistente de un usuario, evitando que deba reautenticarse si recarga la pagina.

* Servicios de Datos Reactivos:

A traves del uso de la libreria RxJS (BehaviorSubjects), los cambios en los programas academicos, la lista de usuarios y las metricas de avance de las facultades se actualizan dinamicamente en tiempo de ejecucion. Estos cambios persisten durante la navegacion activa, pero se reinician a sus valores de prueba preestablecidos si la aplicacion es recargada completamente desde el servidor local (F5).

* Mecanismo de Login de Emergencia:

El servicio de autenticacion intenta opcionalmente consumir un endpoint local de desarrollo. Si dicho endpoint no esta disponible, el sistema intercepta el error de inmediato y aplica una logica de respaldo ('tryMockLogin') que valida las credenciales contra un arreglo estatico de usuarios predefinidos, otorgando acceso inmediato al dashboard correspondiente.

2. Estructura deCodigo y Componentes Clave

El codigo sigue las convenciones recomendadas para aplicaciones modernas de Angular, haciendo uso de Componentes Standalone (independientes) y una arquitectura modular organizada de la siguiente manera:

Archivo / Directorio	Proposito y Funcionamiento en el Prototipo
src/app/app.routes.ts	Define las rutas principales y subrutas de la app, mapeando los dashboards segun el rol del usuario.
src/app/services/auth.service.ts	Controla la sesion del usuario. Maneja la escritura/lectura en localStorage y el fallback de credenciales de prueba.
src/app/services/lider-data.service.ts	Contiene el estado de los programas, facultades e instructores. Implementa las funciones de adicion, edicion y eliminacion simuladas en memoria.
src/app/services/export.service.ts	Permite la descarga de reportes en formato CSV y la impresion/exportacion limpia de la interfaz actual a PDF utilizando estilos de impresion.
src/app/views/login/login.component.ts	Pantalla de acceso. Incluye 'atajos de rol' (botones de ayuda) para iniciar sesion con cuentas preestablecidas de prueba.
src/app/views/dashboards/	Directorio que aloja las interfaces personalizadas para cada rol: administrador, decano, docente y lider de aseguramiento.

3. Flujo de Roles y Acceso al Prototipo

El sistema provee una experiencia adaptada al perfil del usuario. Al iniciar sesion en el prototipo, se pueden emplear las siguientes credenciales preconfiguradas con contraseña estandar '1234':

Administrador de Sistema (admin@ibero.edu.co)

Posee privilegios globales para auditar el estado del sistema, visualizar listas completas de usuarios, su estado (activo/inactivo) y gestionar accesos tecnicos.

Lider de Aseguramiento de Calidad (lider@ibero.edu.co)

Gestiona el catalogo de programas academicos, anade nuevos registros, edita su informacion, asigna colaboradores, monitorea las 51 condiciones del registro calificado y genera reportes de gestion.

Decano (decano@ibero.edu.co)

Perfil directivo con acceso a graficos globales de rendimiento y avance por facultades. Cuenta con herramientas para la exportacion de reportes de cumplimiento y supervision de programas asignados a su facultad.

Docente / Colaborador (docente@ibero.edu.co)

Encargado de la parte operativa: responde a las condiciones especificas asignadas a sus programas, carga evidencias (simuladas) y actualiza el estado de avance del registro calificado.

Mecanismos de Simulacion Importantes

Para validar flujos como la subida de evidencias o modificacion de programas, los formularios operan de forma reactiva. Por ejemplo, al crear un programa en el dashboard del Lider, este aparece de inmediato en la tabla y en las estadisticas globales. Sin embargo, dado que no hay base de datos fisica, al recargar la ventana con F5 o reiniciar el servidor de desarrollo, la base de datos simulada del LiderDataService se reestablecera a su estado por defecto.

4. Guia Completa para Pasar de Prototipo a Produccion

Para transicionar IberomapAngular de un prototipo aislado a una plataforma real con base de datos y servidor back-end, se debe implementar una arquitectura robusta de 3 capas. A continuacion, se explica paso a paso como realizar este proceso y estructurar el sistema:

4.1 Eleccion del Stack y Creacion del Back-End (API REST)

Se debe levantar un servidor central (Back-End) que reciba peticiones HTTP del frontend de Angular. Las opciones de tecnologías recomendadas son:

- Node.js / Express o NestJS (Mantiene la coherencia usando TypeScript/JavaScript).
- Python / FastAPI o Django (Excelente velocidad de desarrollo y manejo de reportes).
- .NET Core / C# o Java Spring Boot (Para entornos institucionales corporativos).

Este servidor contara con routers especializados para '/api/auth' (autenticacion), '/api/programs' (CRUD de programas academicos), '/api/users' (gestion de usuarios) y '/api/conditions' (gestion de las 51 condiciones y archivos de evidencias).

4.2 Modelamiento de la Base de Datos (Relacional - RDBMS)

Toda la informacion simulada en LiderDataService debe almacenarse en un motor de base de datos relacional como PostgreSQL, MariaDB o SQL Server. El modelo de datos debe incluir las siguientes tablas:

*** Tabla de Usuarios (Users):**

id (PK), name, email, password_hash (contrasenas encriptadas con bcrypt), mapRole (admin/lider/decano/docente), faculty (facultad asociada).

*** Tabla de Programas (Programs):**

id (PK), name, faculty, type (Renovacion/Acreditacion), status (en progreso/completado/no iniciado), snies (codigo nacional), conditions_total (51), conditions_completed (calculado de las condiciones).

*** Tabla de Condiciones (Conditions):**

id (PK), program_id (FK), condition_number (1 al 51), name, status (no iniciado/en progreso/completado), assigned_to_user_id (FK a Users, docente encargado).

*** Tabla de Evidencias / Archivos (Evidences):**

id (PK), condition_id (FK), file_name, file_url (enlace de descarga), uploaded_by_user_id (FK), uploaded_at (fecha).

4.3 Modificaciones Requeridas en elCodigo Angular

Para comunicar la interfaz de usuario con la base de datos a traves del back-end, se deben modificar las siguientes secciones del codigo actual de Angular:

1. Sustituir Memoria por HttpClient:

En `src/app/services/lider-data.service.ts`, se eliminaran las declaraciones `BehaviorSubject` con arreglos estaticos en duro. Se inyectara `HttpClient` y se reemplazaran los metodos por llamadas observables REST.

Ejemplo:

```
getPrograms(): Observable<Program[]> {  
  return this.http.get<Program[]>(`${this.apiUrl}/programs`);  
}
```

2. Implementar Guardias de Ruta (Guards):

En `src/app/app.routes.ts`, se deben implementar Guardias de Seguridad (`CanActivate`). Estas impediran que un usuario altere localmente el `localStorage` para suplantar roles (ej. cambiar rol a 'admin' localmente). El guardia consultara el back-end para verificar que el token JWT sea valido y corresponda al rol solicitado.

3. Subida Real de Evidencias (Archivos):

En el modulo del docente, el cargue de evidencias actual solo modifica un contador simulado. Se debe implementar una peticion POST de tipo `FormData` para transferir el archivo adjunto al servidor. El servidor almacenara el archivo en una carpeta segura o en un almacenamiento en la nube (ej. AWS S3) y escribira la URL resultante en la tabla Evidences.

5. Modulos por Desarrollar (Proxima Fase)

Durante la auditoria del estado de desarrollo del frontend actual se identificaron areas incompletas y funcionalidades pendientes para los perfiles de Decano y Administrador, las cuales se definen como tareas por desarrollar para lograr un despliegue completo:

5.1 Modulo de Decano (Pendiente e Integracion)

El modulo actual del Decano cuenta con pantallas y disenos para visualizacion y reportes, pero se encuentra aislado de los servicios reactivos del sistema. Es necesario realizar las siguientes integraciones:

* Conexion Dinamica de Datos:

Actualmente la lista de programas en el modulo del decano esta quemada de forma estatica en los componentes locales (decano-programas.component.ts). Debe conectarse a LiderDataService para reflejar la realidad del sistema en tiempo real.

* Filtrado por Facultad:

El decano debe visualizar unicamente la informacion perteneciente a su Facultad. Se requiere implementar un filtro reactivo basado en el campo 'faculty' asignado al usuario decano logueado.

* Detalles de Programas:

Se debe implementar un modal o navegacion detallada para que el decano pueda dar clic sobre cualquier programa y visualizar el listado de las 51 condiciones, saber exactamente cuales estan listas, que docentes trabajaron en ellas y auditar los archivos/evidencias adjuntas en modo de lectura.

* Consolidacion de Reportes:

La descarga en formato PDF y Excel (decano-reportes.component.ts) solo muestra toasts simulados. Se requiere integrarlos con el ExportService para generar reportes en tiempo real de los programas activos y su porcentaje de avance general.

5.2 Modulo de Administrador (Funciones a Implementar)

El modulo del Administrador (admin-dashboard.component.ts) solo cuenta con la estructura visual de tres paneles generales sin comportamiento operativo. Se definen las funciones obligatorias para cada panel:

* Panel 1: Configuracion General

Permitir la creacion y apertura de nuevos Periodos Academicos (ej. 2026-1, 2026-2). Administracion de parametros institucionales (nombre del rector, decanatos activos, logos de la institucion) y gestion global del catalogo basico de Facultades.

* Panel 2: Seguridad y Roles

Control exhaustivo del listado de usuarios del sistema con permisos de edicion (CRUD completo de usuarios). Asignacion y cambio de Roles de manera interactiva, visor de bitacora / logs de auditoria (rastreo de cambios de evidencias y programas) y forzar cierre de sesiones activas.

* Panel 3: Mantenimiento de Datos

Modulo para la creacion manual y descarga de backups de la base de datos completa en formato JSON/SQL. Depuracion de registros huérfanos y la interfaz de sincronizacion / conexion con sistemas academicos externos de la universidad (ej. Banner o SIU) para importar estudiantes y programas automaticamente.

Repositorio del Proyecto:

<https://github.com/polarsama/iberomap-angular>

© 2026 Corporacion Universitaria Iberoamericana